



Programmiersvorkurs

Einführung in C# - Tag 2

Inhaltsübersicht



Tag 1	Variablen, Datentypen, Arithmetik, Konvertierung, Zustände, Visual Studio Demo
Tag 2	Boolsche Ausdrücke, Kommentare, If-Abfragen, Switch-Case
Tag 3	Arrays, (Do-)While-Schleife, For-Schleifen, Debugging
Tag 4	Methoden, Klassen

Kommentare

- Als „Erklärung“ von Code-Segmenten
- Schnelles Deaktivieren von Code für Debugging
- Haben **keinen** Einfluss auf das Programm (wird vom Compiler ignoriert)

```
1 int number; //Kommentar
2
3 //number = 3;
4
5 number = 12;
```

Kommentare - Best Practices

- Kommentare knapp halten
- Kommentare sollten sinnvoll sein
- Code sollte sich (optimalerweise) selbst erklären

```
1 int number = 1;
2 int number2 = 1;
3
4 int result;
5
6 result = number + number2;
7 number = number2;
8 number2 = result;
```

```
1 //fibonacci sequence
2 int first = 1;
3 int second = 1;
4
5 int nextFibonacci;
6
7 //calculate the first iteration of the fibonacci sequence
8 nextFibonacci = first + second;
9 first = second;
10 second = nextFibonacci;
```

Kommentare

Zeilenkommentar:

```
1 int number; //Kommentar
2
3 //number = 3;
4
5 number = 12;
```

Blockkommentar:

```
1 int number;
2
3 /*
4  number = 3;
5  number += 8;
6  */
7
8 number = 12;
9 Console.WriteLine(number);
```

TODO-Kommentar:

```
1 string title = "Lorem ipsum dolor sit amet..." //TODO: replace this
```

Boolscher Ausdruck

- Wahrheitswert – kann **true** oder **false** sein
- Ergebnis eines Vergleichs
- Kann in einer Variable vom Typ „**bool**“ gespeichert werden

```
1 bool isHungry = false;  
2 isHungry = true;  
3  
4 bool isGreater = (2 > 4);
```

Boolscher Ausdruck

>	größer
<	Kleiner
>=	größer gleich
<=	kleiner gleich
==	gleich
!=	ungleich

```
1 bool isGreater = (3 > 29);
2
3 bool isLesser = (4 < 14);
4
5 bool isGreaterOrEqual = (3 >= 2);
6
7 bool isLesserOrEqual = (45 <= 45);
8
9 bool isEqual = (1 == 5);
10
11 bool isNotEqual = (3 != 5);
```

Verknüpfungen

Verknüpfung	Sprich	Bedeutung
$a \ \& \ b$	a und b	a und b true $\rightarrow$ Verknüpfung true
$a \ \ b$	a oder b	a oder b true $\rightarrow$ Verknüpfung true
$a \wedge b$	entweder a oder b	nur a oder b true $\rightarrow$ Verknüpfung true
$!a$	nicht a	a false $\rightarrow$ Verknüpfung true

Oft sieht man auch **&&** und **||**, diese Verknüpfungen sind für uns erstmal identisch!

```
1 bool isRaining = true;
2 bool isCold = true;
3
4 bool isBadWeather = isRaining && isCold;
```

Verknüpfungen

```
1 bool isOnCampus = true;
2 bool isInLibrary = false;
3
4 bool isStudying = isOnCampus || isInLibrary;
```

```
1 bool userLoggedIn = true;
2 bool userAdmin = false;
3
4 bool isNormalSession = userLoggedIn ^ userAdmin;
```

```
1 bool hasRecentlyEaten = true;
2
3 bool isHungry = !hasRecentlyEaten;
```

Verk.	Bedeutung
$a \& b$	a und b
$a b$	a oder b
$a \wedge b$	entweder a oder b
$!a$	nicht a

Rangfolge der Operatoren



++, --	Inkrement und Dekrement
!	Negation
*, /, %	Multiplikation, Division, Modulo
+, -	Addition und Subtraktion
==, !=, <=, >=	Vergleich
&	Und
	Oder
&&	Kurzschlussoperator Und
	Kurzschlussoperator Oder
=	Zuweisung

Unäre Operatoren

Rechenzeichen

Binäre Operatoren

Zuweisung

Rangfolge der Operatoren

```
1 int number = 3;  
2 bool isComplex = ++number == 4 | !false & 103 <= 19;
```

Klammern setzen!

```
1 int number = 3;  
2 bool isComplex = ((++number) == 4) | ((!false) & (103 <= 19));
```

++, --	Inkrement und Dekrement
!	Negation
*, /, %	Multiplikation, Division, Modulo
+, -	Addition und Subtraktion
==, !=, <=, >=	Vergleich
&	Und
	Oder
&&	Kurzschlussoperator Und
	Kurzschlussoperator Oder
=	Zuweisung

Beispiel

Sportwagen

Unsere Definition eines Sportwagens:

- Maximal 2 Türen, keine Rücksitze
- Höchstgeschwindigkeit von min. 280 km/h
- **oder** Höchstgeschwindigkeit von mindestens 230 km/h und mindestens 300 PS



```
1 (doors <= 2)
2 && !hasBackseats
3 && ((maxSpeed >= 280) || (maxSpeed >= 230 && powerInPS >= 300))
```

Verlgeiche (Gleitkommazahlen) HKA

```
1 double a = 0.6 + 0.7;  
2 Console.WriteLine(a == 1.3);
```

FALSE

Name	Value
 a	1.2999999999999998

Rundungsfehler tritt auf!

- Da C# Gleitkommazahlen als „Zahl * 10erPotenz“ speichert
- Effizient aber teils ungenau

Verlgeiche (Gleitkommazahlen)

```
1 double a = 0.6 + 0.7;  
2 Console.WriteLine(a == 1.3);
```

TRUE

Name	Value
 a	1.2999999999999998

Lösung 1

- Integer benutzen

Verlgeiche (Gleitkommazahlen)

```
1 decimal a = 0.6m + 0.7m;  
2 Console.WriteLine(a == 1.3m);
```

TRUE

Name	Value
 a	1.3

Lösung 2 (klappt manchmal)

- Gleitkommatyp „decimal“ benutzen

Verlgeiche (Gleitkommazahlen)



```
1 double a = 0.6 + 0.7;  
2 bool isAlmostEqual = Math.Abs(a - 1.3) < 4.5E-10;  
3  
4 Console.WriteLine(isAlmostEqual);
```

TRUE

Math.Abs()
berechnet den
absoluten Wert
einer Zahl

$$\begin{aligned} 4.5\text{E-}10 &= 4.5 * 10^{-10} \\ &= 0.00000000045 \end{aligned}$$

Lösung 3

- Überprüfen, ob *fast* gleich
- „true“, wenn Differenz der Zahlen unterhalb von einem sehr kleinen Wert liegt

If-Abfragen

Code NUR dann ausführen, **wenn** eine **Bedingung** (bool) eintrifft!

- Wenn **Bedingung** zutrifft, dann wird der code innerhalb der Klammern ausgeführt (**Anweisung1**)

```
1 if (Bedingung)
2 {
3     Anweisung1;
4     //...
5 }
```

If-Abfragen

Bsp.:

Wenn es regnet, soll das Programm mir einen Text ausgeben.

```
1 bool isRaining = true;
2
3 if (isRaining)
4 {
5     Console.WriteLine("you'll need an umbrella!");
6 }
```

If-Abfragen

Bsp. 2: Bezahlung

```
1 int moneyAvailable = 54;
2 int costs = 13;
3
4 if (moneyAvailable >= costs)
5 {
6     moneyAvailable -= costs;
7     Console.WriteLine("payment accepted");
8 }
```

If-Else

- „if“ lässt sich erweitern mit „if-else“
- Wenn **Bedingung1** nicht zutrifft, dann mache „else“

```
1 if (Bedingung)
2 {
3     Anweisung1;
4     Anweisung2;
5     // ...
6 }
7 else
8 {
9     // ...
10 }
```

If-Else

Bsp.:

Wenn wir Mate haben, nimm dir eine, wenn nicht, Pech gehabt!

```
1 if(weHaveMate)
2 {
3     Console.WriteLine("Nimm ne Mate");
4 }
5 else
6 {
7     Console.WriteLine("Pech gehabt!");
8 }
```

If-Else

- „if-else“ lässt sich nochmal erweitern
- Wenn **Bedingung1** nicht zutrifft, dann überprüfe **Bedingung2**, wenn **Bedingung2** nicht zutrifft, dann mache **else**

Mate-Bsp.:

```
1 if(weHaveMate)
2 {
3     Console.WriteLine("Nimm ne Mate");
4 }
5 else if (weHaveCoffee)
6 {
7     Console.WriteLine("Nimm nen Kaffee");
8 }
9 else
10 {
11     Console.WriteLine("Pech gehabt!");
12 }
```

If-Abfragen

- Verschachtelung möglich!
- Verschachtelung möglichst vermeiden :[

Verschachtelung-Bsp.:

```
1 bool isTree = false;
2
3 if(isTall)
4 {
5     if(isGreen)
6     {
7         if(hasLeaves)
8         {
9             isTree = true;
10        }
11    }
12 }
```

Quellen und Mitwirkende



Original von Anna Roes

Überarbeitet 2011 & 2015 & 2016 von Moritz Grimm

Überarbeitet 2013 von Sebastian Wörner

Überarbeitet 2016 von Sarah Kohler

Überarbeitet 2018 von Eva Wolkwitz

Überarbeitet 2019 von Christian Wernet

Überarbeitet 2020 von Jana Becker

Überarbeitet 2021 von Tobias Hnyk

Überarbeitet 2025 von Simon Weinhardt